Project Information			
Group ID	48		**Semester** *(WS2?)*
1. Student Name	Divyansh Sharma		WS5
2. Student Name	Ankita Samal		WS2
3. Student Name	Jasdeep Singh		WS2
4. Student Name	Bhavika Balapota		WS2
Project Title	Decentralized Robot Task Allocation		

Project Description	=> We use a peer-to-peer architecture where every robot runs the same code. There is no fixed server, so any robot can take over any role. => One robot is elected as the dispatcher. It works as a sequencer: it opens and closes the bidding for each task and gives every bid a global sequence number. All other robots act as bidders and keep a full replica of the task state (open tasks, best bids, bid history and assignments). => Each robot consists of the following modules: discovery, heartbeat, election, sequencer (only active on the dispatcher), ordered delivery with a hold-back queue, task state, bid scorer and a small command-line interface. => For communication we use UDP multicast for discovery, UDP unicast with our own reliability mechanism for all protocol messages, and TCP only when a new robot needs to receive the current state.

Architectural Model	=> When a robot starts, it chooses a random ID, joins a multicast group and sends a HELLO message three times to make sure it is not lost. => All robots that are already running answer with their ID, the current dispatcher and the current epoch number. Based on these answers, each robot builds its own membership view. This view comes only from discovery and never from a static configuration file. => Robots repeat their HELLO every 5 seconds. If a robot is not heard from for 15 seconds, it is removed from the view. => If a new robot gets no answer within 2 seconds, it assumes it is alone and becomes the dispatcher of a fleet with one robot. => Before a new robot takes part in task allocation, it first receives the current state from the dispatcher. This way robots can join and leave the fleet at any time, and the only fixed value in the system is the multicast address.

Dynamic Discovery	=> When a robot starts, it chooses a random ID, joins a multicast group and sends a HELLO message three times to make sure it is not lost. => All robots that are already running answer with their ID, the current dispatcher and the current epoch number. Based on these answers, each robot builds its own membership view. This view comes only from discovery and never from a static configuration file. => Robots repeat their HELLO every 5 seconds. If a robot is not heard from for 15 seconds, it is removed from the view. => If a new robot gets no answer within 2 seconds, it assumes it is alone and becomes the dispatcher of a fleet with one robot. => Before a new robot takes part in task allocation, it first receives the current state from the dispatcher. This way robots can join and leave the fleet at any time, and the only fixed value in the system is the multicast address.

Fault Tolerance	=> We focus on crash-stop failures. => The dispatcher sends a heartbeat every 2 seconds. If the robots miss three heartbeats in a row (6 seconds), they assume the dispatcher has crashed and start an election. => If a normal bidding robot crashes, it is simply removed from the membership view and the task allocation continues without interruption. => If the dispatcher crashes, a new dispatcher is elected. Before continuing, it collects the sequence numbers from all remaining robots so that no operation is lost or applied twice, and then it continues with the remaining bidding time. => If a bid gets lost in the network, the robot sends it again with the same bid ID until it appears in the ordered stream. Because of the bid ID, a bid can never be counted twice. => A robot only learns whether its bid was accepted when it delivers that bid itself from the ordered stream. Since every operation that a surviving robot has delivered is re-sent by the new dispatcher, accepted bids are never lost. => A robot that restarts after a crash joins as a new member and first receives a snapshot of the current state over TCP.

Voting	=> For the election we use the Bully algorithm, extended with epoch numbers to make it more robust. => An election starts when there is no dispatcher at startup, when the dispatcher stops sending heartbeats, or when a robot with a higher ID than the current dispatcher joins the fleet. => The robot that starts the election sends an ELECTION message to all robots with a higher ID. If a higher robot answers, it takes over the election. If no one answers within 2 seconds, the robot becomes the new dispatcher and announces this with a COORDINATOR message. If no COORDINATOR message arrives within 5 seconds after an answer, the election is started again. => Every message carries an epoch number, and messages with an older epoch are ignored. This means that an old dispatcher that comes back after a crash cannot disturb the system; it sees the higher epoch and steps down. => If several robots start an election at the same time, the robots with lower IDs stop, so in the end the robot with the highest live ID always wins.

Reliable Ordered Multicast	=> We implement total ordering, with the dispatcher acting as sequencer. => Robots send their bids to the dispatcher. The dispatcher gives each bid a global sequence number and multicasts it to all robots, including itself. => Every robot delivers the operations strictly in the order of the sequence numbers. Messages that arrive too early are stored in a hold-back queue. If a robot notices a gap, it sends a NACK and the dispatcher sends the missing message again from its history buffer. The heartbeats also contain the latest sequence number, so even a lost last message is detected. => The bidding for a task is closed with a CLOSE operation that is ordered just like the bids. Only bids that come before the CLOSE in the order are counted. Because of this, the result is clear without needing synchronized clocks. => We need total ordering because bids from different robots are sent independently at the same time. With FIFO or causal ordering, robots could process these bids in a different order and choose different winners, which would send two robots to the same task. => After a dispatcher crash, the new dispatcher asks all robots for the highest sequence number they have delivered, re-sends any missing operations and continues from there.

System Architecture Design

Include a clear diagram of your system architecture (key components and connections between them).

